LAB 4

Simple Python Database Connections

Name: Sidra Tul Muntaha

Registration Number : 23-BSCS-084

GitHub Username: sidra2244

Submission Repository: DevOps

Submission Path: Labs/Lab04/

Date: 21 October 2026

Fork Repository (Database) URL:

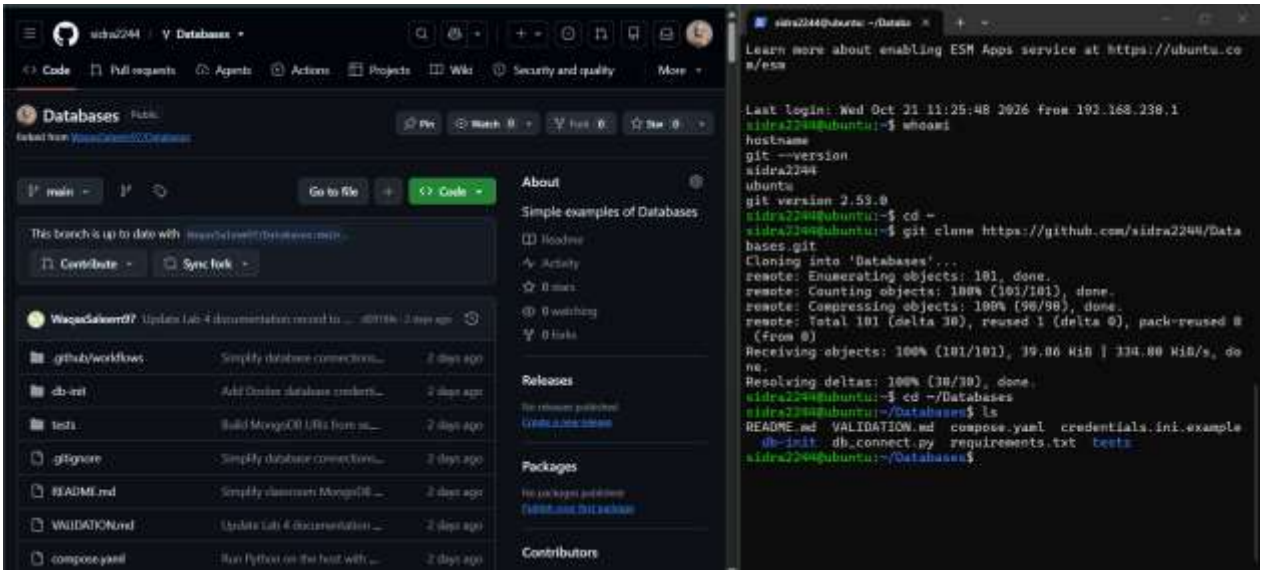
https://github.com/sidra2244/Databases.git

Submission Repository URL:
https://github.com/sidra2244/DevOps/tree/main/Labs/Lab04

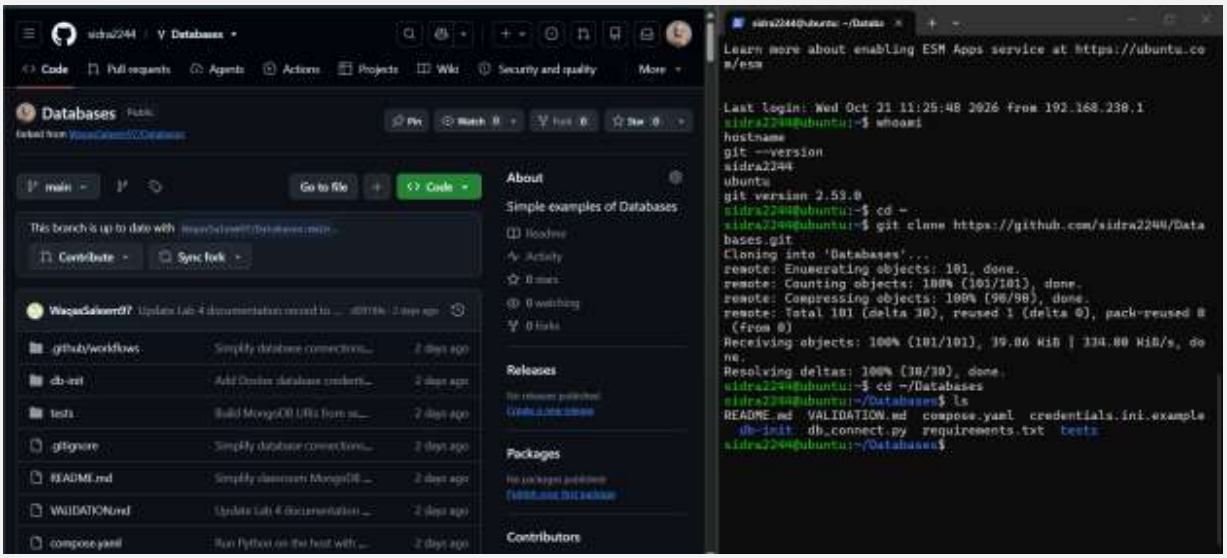
Repository and Environment Setup

The starter repository, Databases, was forked on GitHub and cloned to the Ubuntu environment. The DevOps repository was used as the submission repository. Python dependencies were installed in a virtual environment, and Docker Compose was used to configure the database services.

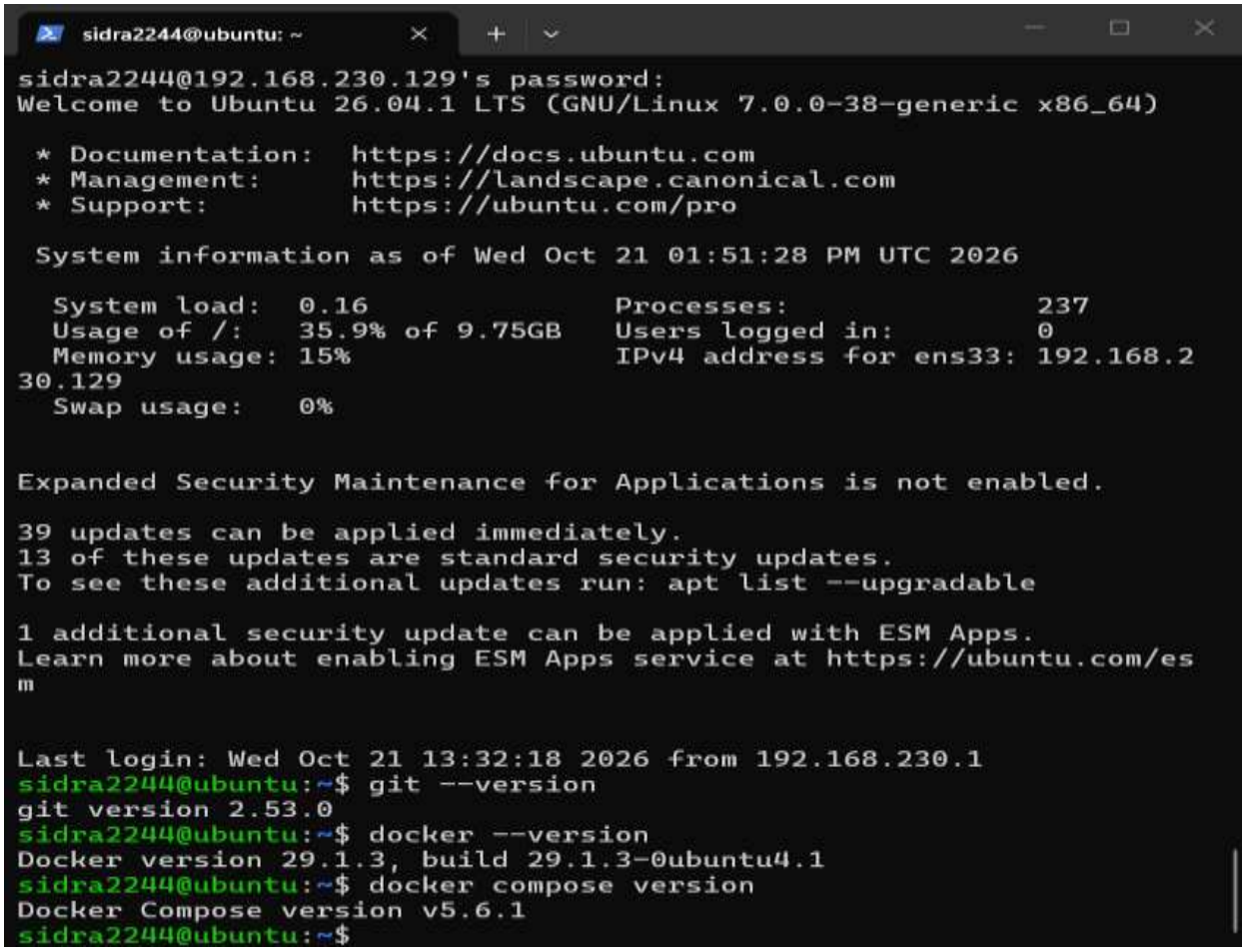
Screenshot: databases_fork.png



Screenshot: databases_clone.png



Screenshot: docker_installation.png

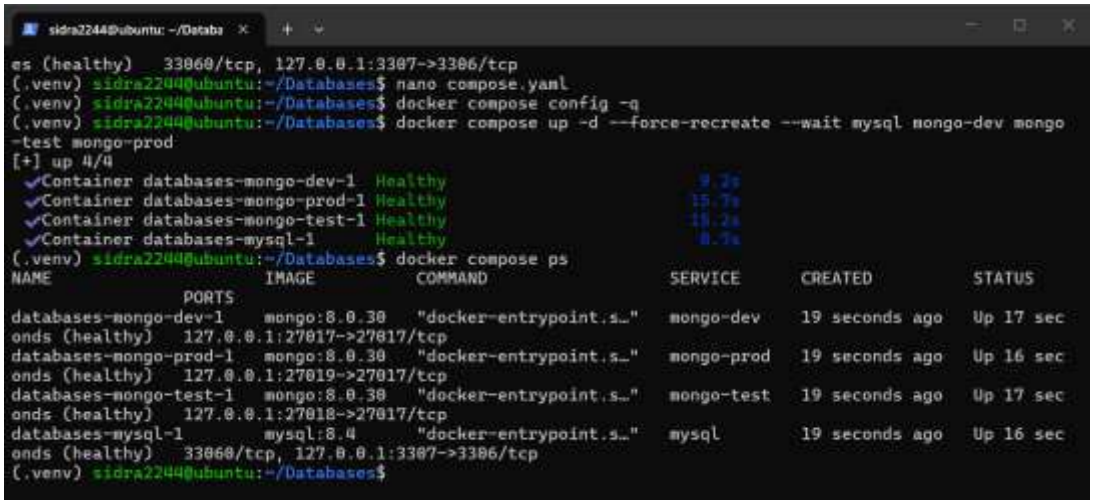


Docker Database Services and Ports

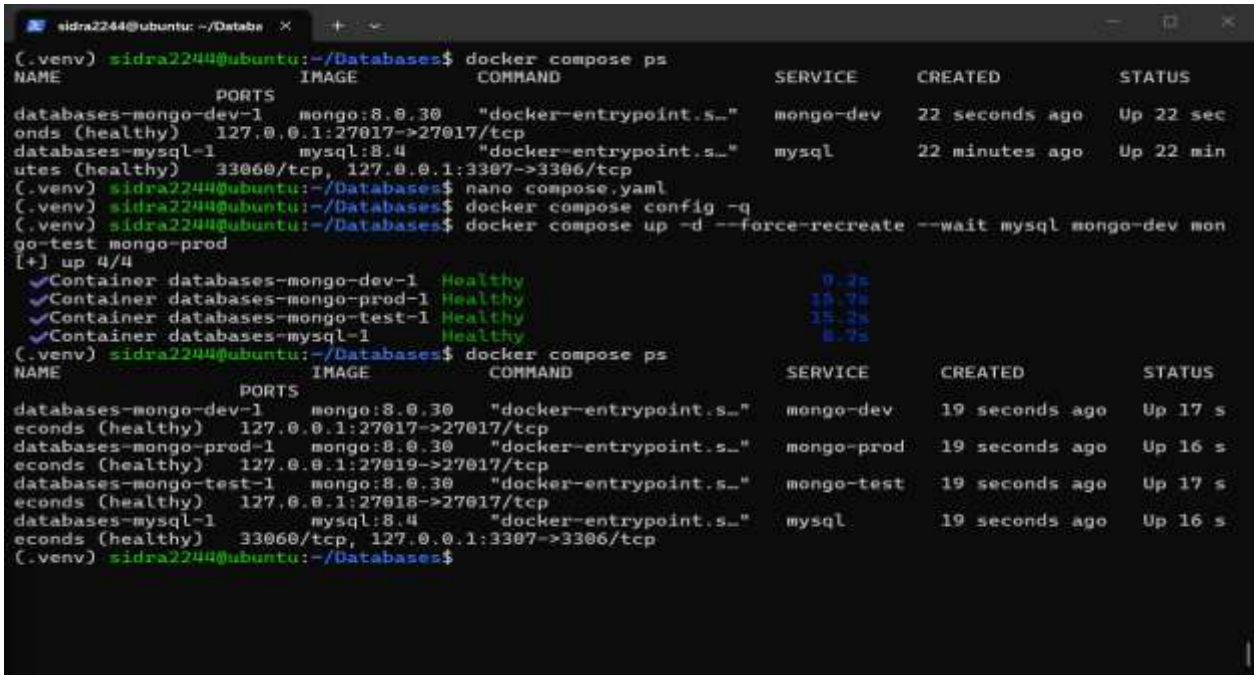
Docker Compose was used to run one MySQL service and three MongoDB environments: development, testing, and production simulation. Each service was mapped to a localhost port so that the Python program could connect from the Ubuntu host.

Service	Host port	Database	Purpose
MySQL	3307	school_mysql	MySQL connection exercise
MongoDB dev	27017	school_dev	Development environment
MongoDB test	27018	school_test	Testing environment
MongoDB prod	27019	school_prod	Production simulation

Screenshot: database_services.png



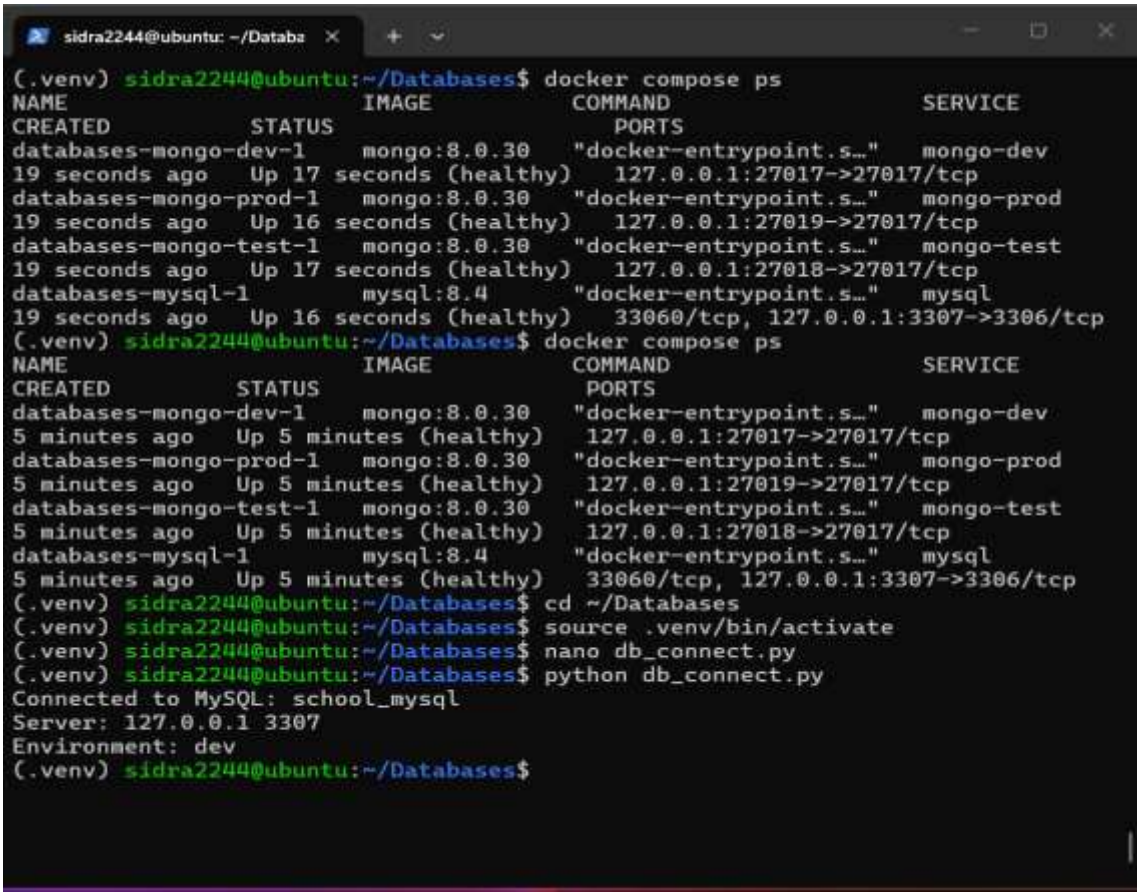
Screenshot: database_ports.png



MySQL Connection Using Hard-Coded Settings

The first Python connection exercise used mysql-connector-python to connect to the school_mysql database on 127.0.0.1:3307. A simple query read the environment value from the environment_info table. The screenshot should show a successful connection and the returned environment.

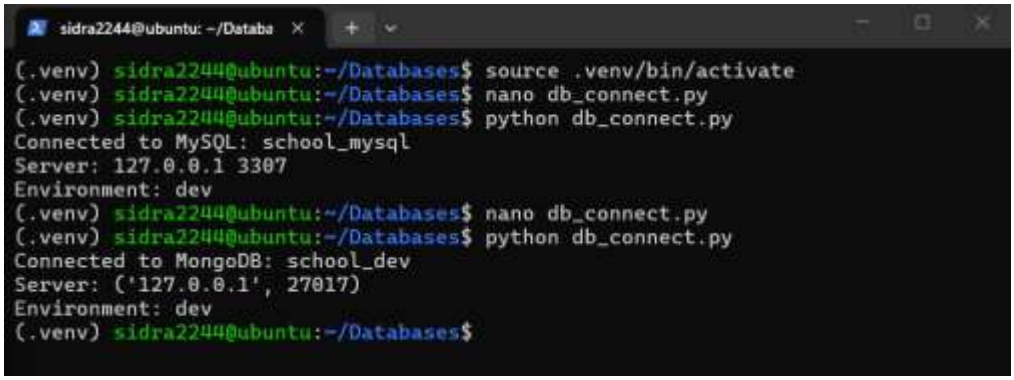
Screenshot: mysql_hardcoded_connection.png



MongoDB Connection Using Hard-Coded Settings

The MongoDB example used PyMongo to connect to the development database, school_dev, at 127.0.0.1:27017. The program pinged the server and retrieved the environment label from the environment_info collection.

Screenshot: mongo_hardcoded_connection.png

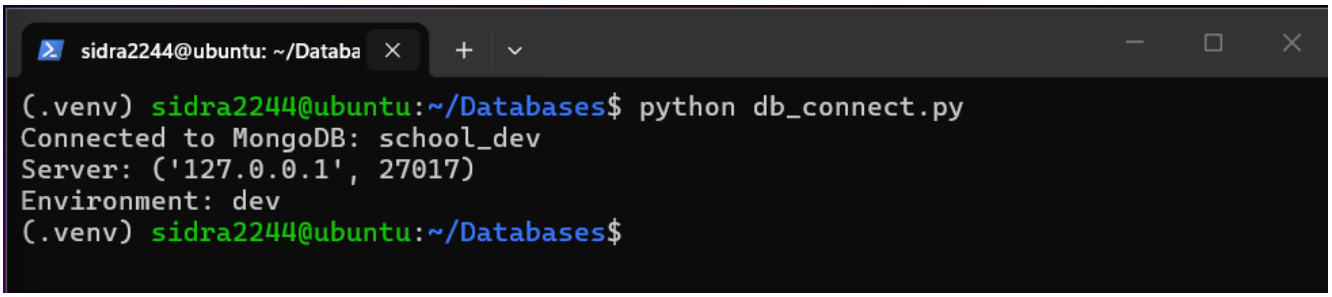


MongoDB Connection Using Environment Variables

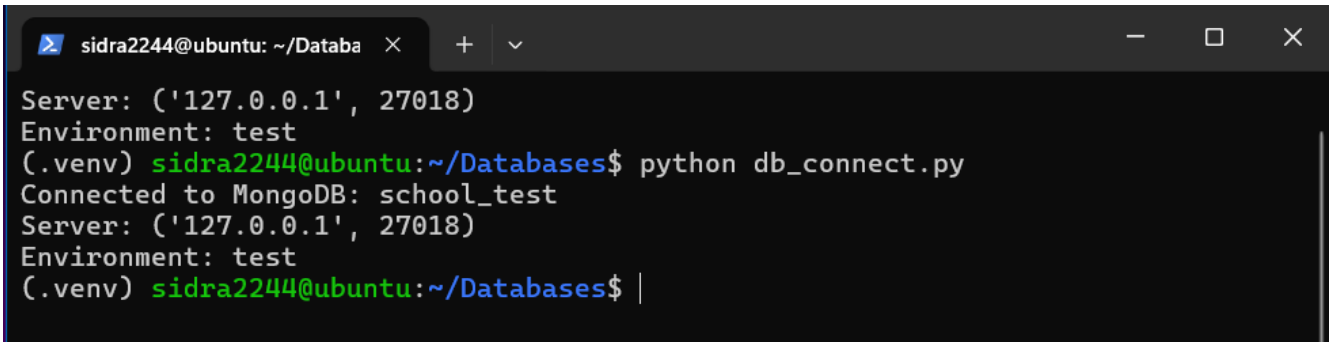
The connection settings were moved into environment variables instead of being directly embedded in the Python source code. MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT, and

MONGO_DATABASE were set for each environment. The program checked that all required values were present before connecting.

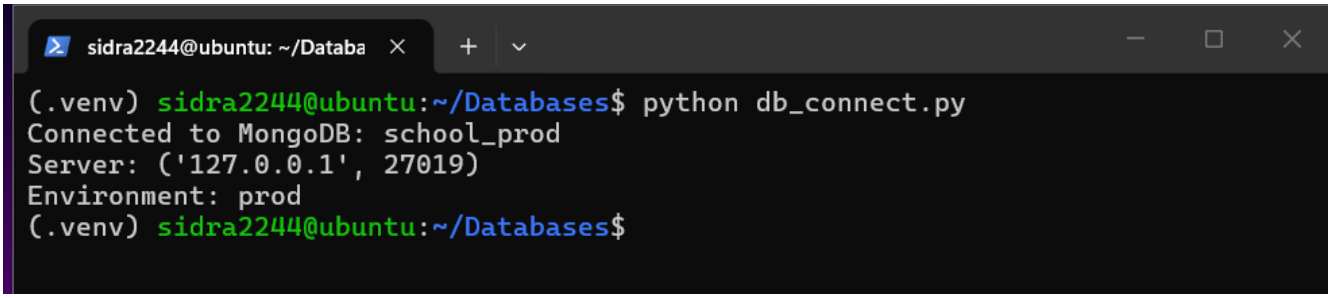
Screenshot: mongo_env_dev.png



Screenshot: mongo_env_test.png



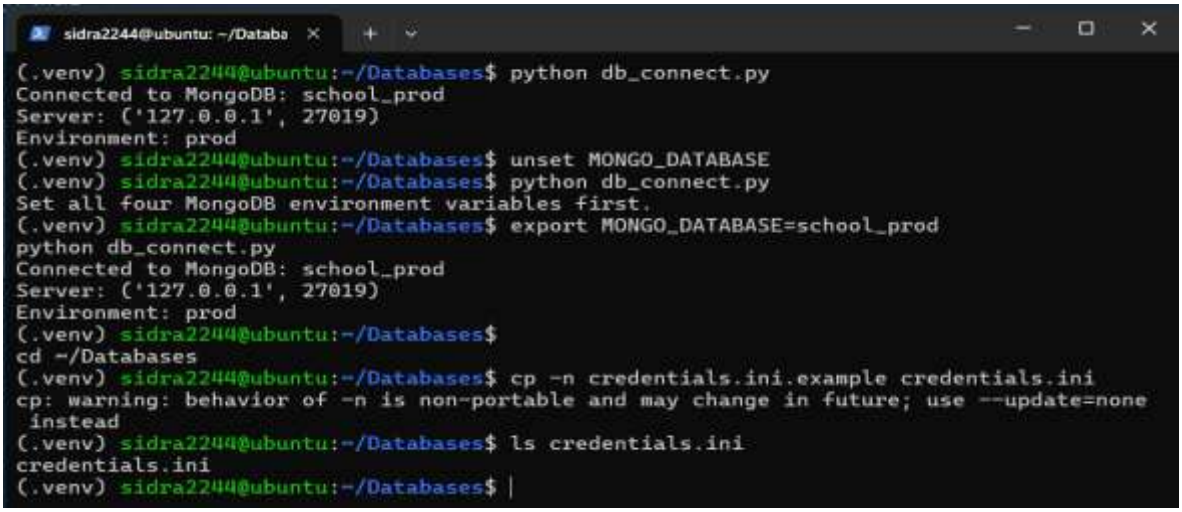
Screenshot: mongo_env_prod.png



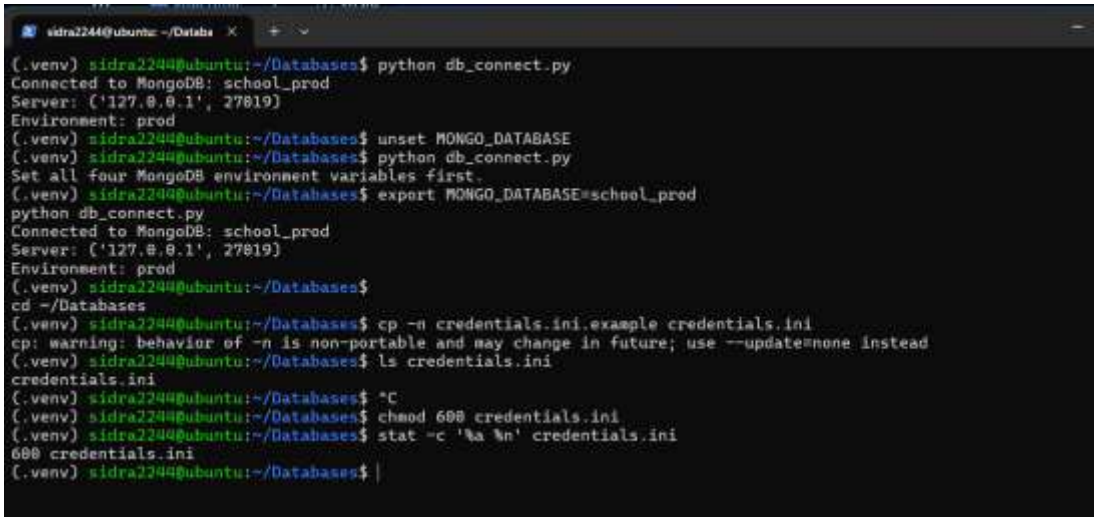
Credentials File and File Permissions

A local credentials.ini file was created from the provided example and protected with chmod 600. This permission allows the file owner to read and write the file while denying access to group and other users. The credentials file should remain local and must not be committed to Git.

Screenshot: credential_files_created.png



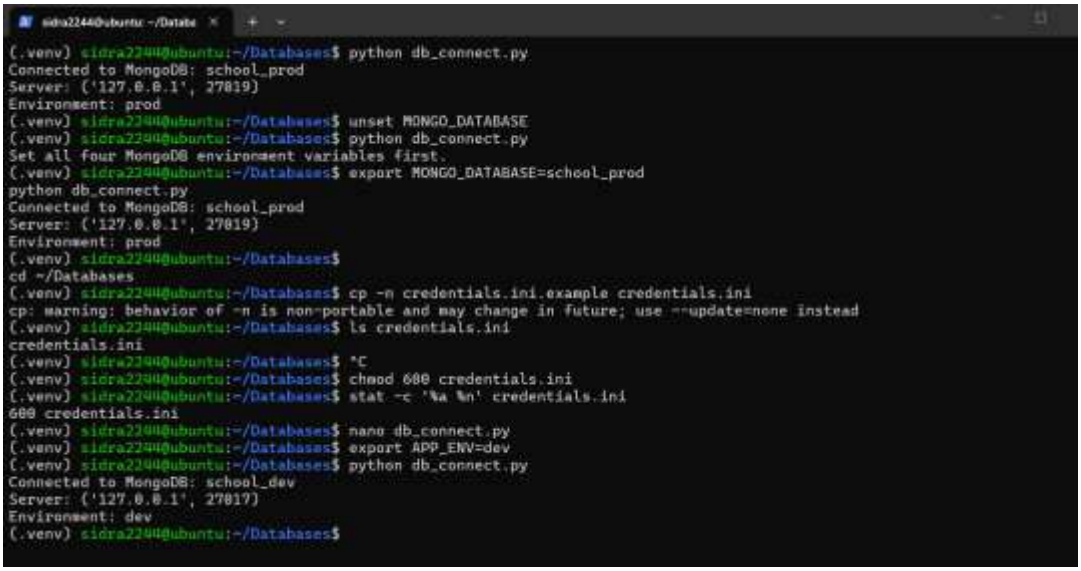
Screenshot: credential_file_permissions.png



MongoDB Connection Using the Credentials File

The final program used Python's configparser to read credentials.ini. APP_ENV selected the target section (dev, test, or prod); if APP_ENV was not set, the program defaulted to dev. The selected credentials were then used to connect to the corresponding MongoDB service.

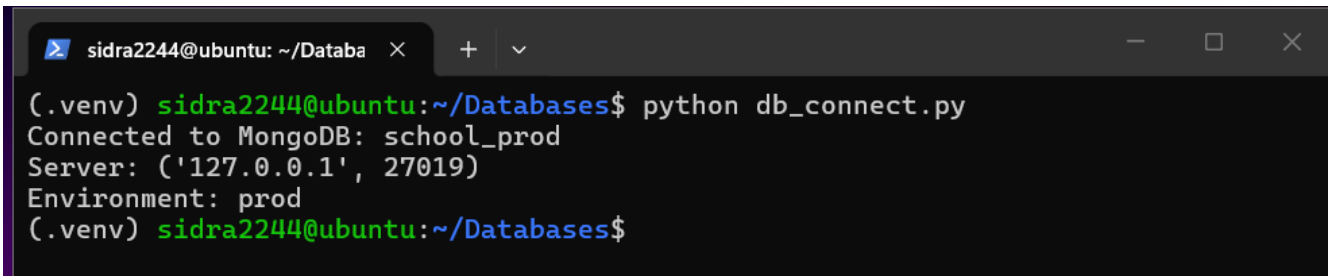
Screenshot: mongo_file_dev.png



Screenshot: mongo_file_test.png



Screenshot: mongo_file_prod.png

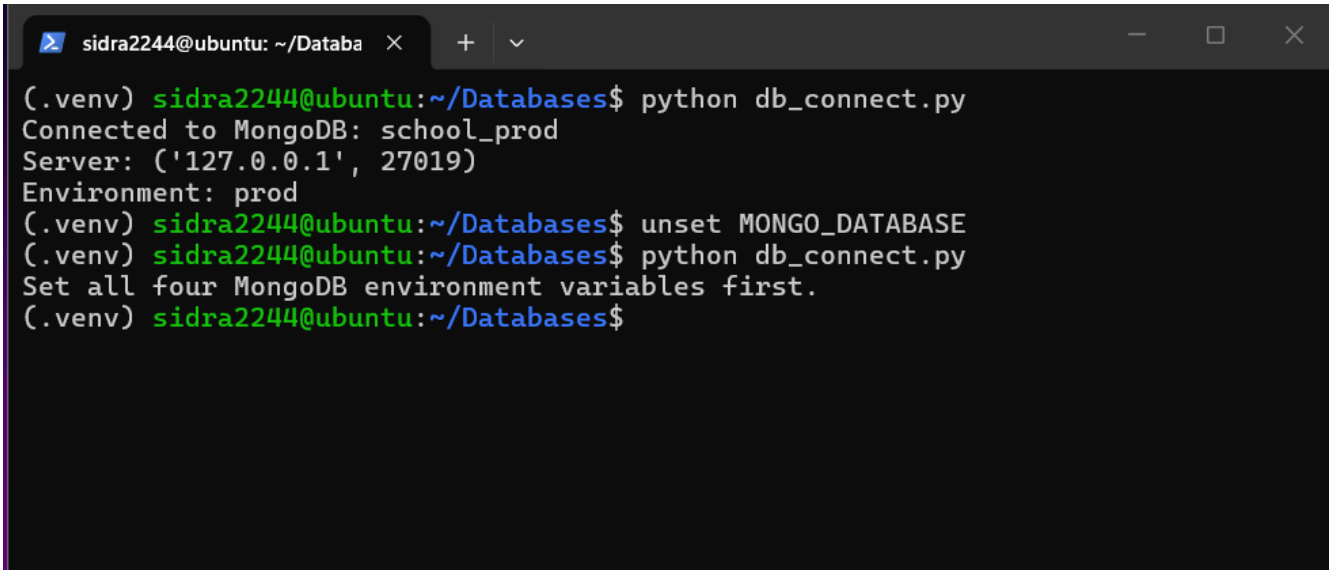


Exam Questions and Observations

Q1. What happens when a required environment variable is missing?

When MONGO_DATABASE was unset, the program stopped before attempting a database connection and displayed a validation message indicating that all four MongoDB environment variables must be set. This makes configuration errors easier to identify.

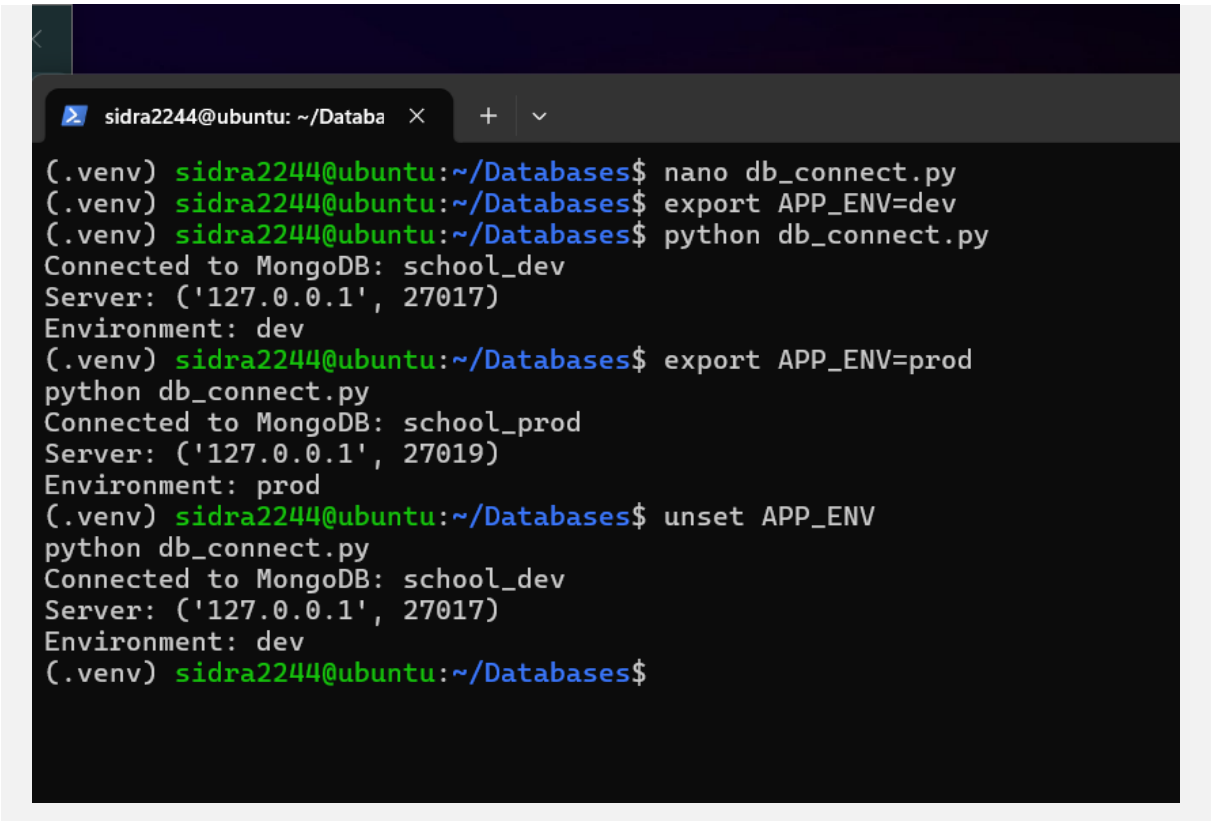
Screenshot: Q1_missing_environment.png



Q2. Which source supplies the connection settings in the file-based version?

The final version reads the selected environment section from credentials.ini. APP_ENV selects the section. If APP_ENV is unset, the code uses dev as its default environment.

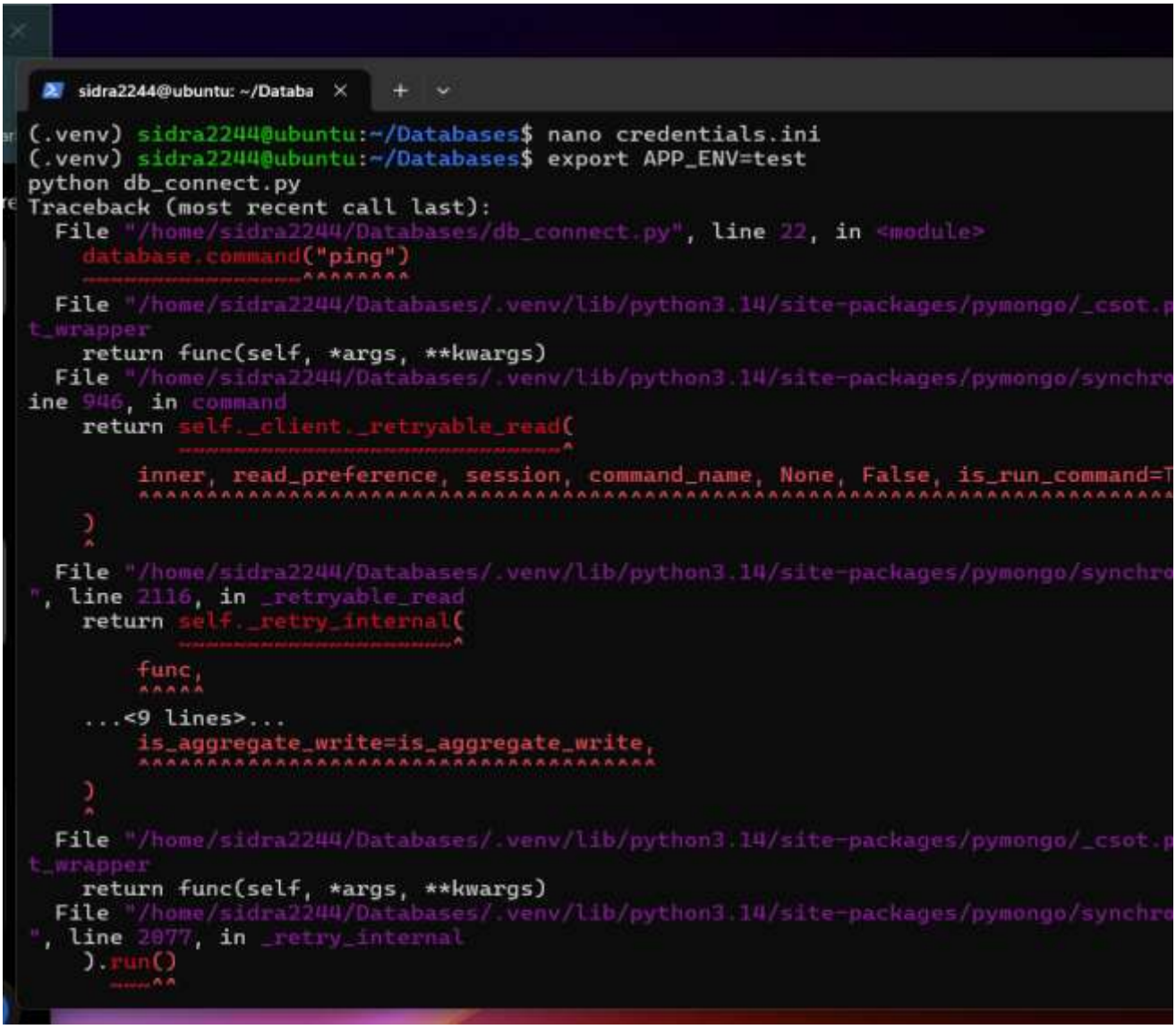
Screenshot: Q2_file_source_selection.png



Q3. What happens when the password in the selected credentials file is wrong?

With an incorrect password in the test section, MongoDB authentication fails and the program cannot complete its ping operation. The original credentials file was then restored from its backup and its permissions were reset to 600.

Screenshot: Q3_wrong_file_password.png



Conclusion

This lab demonstrated database connectivity from Python using MySQL Connector and PyMongo, Docker-based service configuration, environment-specific connection settings, and a local credentials file with restricted permissions. The exercises also demonstrated how missing configuration and incorrect credentials affect connection behavior.